Weakly Supervised Litter Localization using VGG-16 and Class Activation Maps

Simone Vaccari
Artificial Intelligence for Science and Technology
Università degli Studi di Milano-Bicocca
Id. 915222
s.vaccari10@campus.unimib.it

***Abstract*— This work explores Weakly Supervised Object Localization for multi-label litter detection using a fine-tuned VGG-16 model on the TACO dataset. Grad-CAM and LayerCAM are used to generate class activation maps, from which bounding boxes are extracted to localize waste objects. Key challenges, including small object sizes, dataset imbalance, and limited samples, are examined and partially addressed. Synthetic data generation improves performance, and LayerCAM from intermediate layers proves more effective for precise localization.**

***Index terms*—WSOL, Litter Detection, Litter Localization, TACO Dataset, VGG-16, Grad-CAM, LayerCAM, XAI**

I. Introduction

Littering poses a significant environmental and economic challenge across the world, with substantial impacts on communities and ecosystems. According to the Keep America Beautiful 2020 National Litter Study [1], approximately 50 billion pieces of litter are present on U.S. roadways and waterways, equating to over 2,000 items per mile. In Europe, approximately 14 million tonnes of litter are collected annually from the streets, with cleanup costs reaching up to €13 billion [2]. Cigarette butts are the most frequently littered items, accounting for a significant portion of this debris. Plastics, including food packaging, bottles, and bags, constitute about 30% of litter, posing threats to wildlife and contributing to pollution. In Italy, the situation is particularly concerning. Studies by ENEA [3] indicate that over 80% of waste on Italian beaches is composed of plastic, posing serious threats to both the environment and human health. These statistics underscore the critical need for effective litter detection and localization strategies to mitigate environmental impacts and reduce economic burdens.

Weakly Supervised Object Localization (WSOL) offers a promising solution to address the challenges of litter detection and localization, especially in scenarios where obtaining extensive labeled data is impractical. WSOL is the task of identifying and localizing objects in images using only image-level labels, such as class categories, rather than detailed annotations like bounding boxes. This significantly reduces the need for expensive and labor-intensive manual labeling, which can be particularly prohibitive for large-scale datasets.

In the context of waste detection, WSOL is especially advantageous as it enables the training of deep learning models with minimal supervision while still providing useful insights. By leveraging techniques like Class Activation Maps (CAMs), WSOL can generate visual explanations that highlight regions of interest corresponding to specific classes, facilitating accurate localization of litter. This approach not only reduces the cost and complexity of data collection but also accelerates the development of automated systems for waste monitoring and cleanup, making it a key tool in fighting the pervasive issue of littering.

In this project, I explore the localization performance of a fine-tuned VGG-16 network trained on the TACO dataset for a multi-label classification task. Specifically, the trained network is used to generate CAMs using the Grad-CAM and LayerCAM methodologies, which are then used to extract bounding boxes for waste localization.

II. Dataset Description and Analysis

The TACO dataset [4] consists of 1,500 high resolution images of waste in the wild, collected from various environments such as woods, streets, parks, and beaches. Litter instances are segmented and labeled, resulting in a total of 4,784 annotations across 60 different waste categories. The images vary in size and are mostly taken by mobile phones. Figure 1 shows the image area distribution. As we can see, the images in the dataset are relatively large, with a median size of approximately 8 megapixels (roughly 4000x2000), thus necessitating proper resizing. In particular, a few outliers with extremely large image sizes are identified and

subsequently removed, leaving a temporary dataset of 1,480 images.

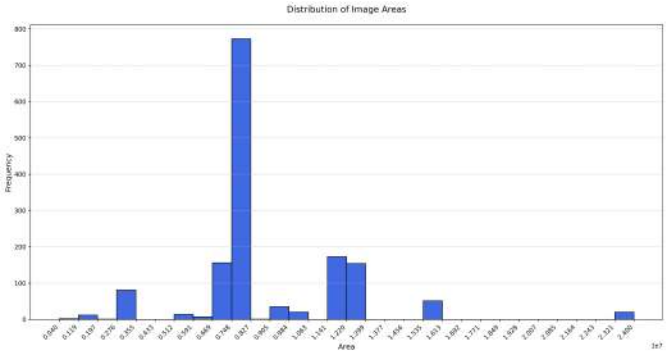


Figure 1: Image area distribution.

A visual inspection of the dataset reveals that the majority of waste objects are significantly smaller compared to the overall image dimensions. To quantify this observation, Figure 2 presents a zoomed-in view of the distribution of relative bounding box areas within the range [0, 0.02], confirming that most bounding boxes occupy only a small fraction of the total image area.

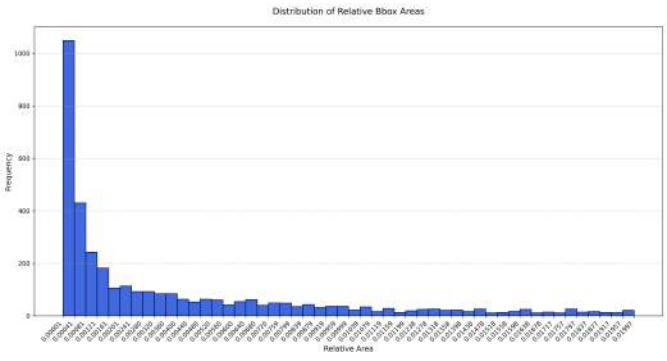


Figure 2: Relative bounding box area distribution in the range [0, 0.02].

Figure 3 displays two examples of very small bounding boxes. As it is unlikely that the network will be able to extract meaningful features from such small objects (which may instead be treated as noise), it is best to remove them. This is especially relevant considering that these small objects often appear in images alongside larger ones, which are more likely to attract the network's focus. However, it is crucial to avoid eliminating annotations that are sufficiently large for the model to recognize. Removing such annotations could lead to inconsistencies, as the network might correctly classify an image but still be penalized due to the absence of the corresponding annotation.

Figure 3: Examples of very small bounding boxes (in red).

The threshold for the minimum accepted relative area is set to 0.002 after analyzing various examples of smaller bounding boxes and considering the relative area distribution in Figure 2. This operation effectively removes all annotations with a bounding box area smaller than 0.2% of the corresponding image area, accounting for 42% of all annotations. While a significant number of small bounding boxes are preserved, selecting a higher threshold is not suggested, as a substantial portion of the data has already been filtered out.

Given the complexity of the task and the dataset, I follow a similar approach to that used in the TACO paper by merging similar classes into 10 macro-categories. One of these, *Other*, consists of a wide variety of litter types. This simplified version of the dataset is referred to as TACO-10.

A slight modification is made to the original TACO-10 dataset: the class *Pop tab* is removed due to its underrepresentation and the fact that, in most cases, the object appears attached to a can, which already belongs to another supercategory. In its place, a new class, *Carton*, is introduced by aggregating various carton-based objects, such as egg cartons, drink cartons, and pizza boxes.

The final ten classes are as follows: *Bottle*, *Bottle cup*, *Can*, *Carton*, *Cigarette*, *Cup*, *Lid*, *Other*, *Plastic bag + wrapper*, and *Straw*.

The next step involves discarding images with an excessive number of annotations. While this approach may reduce the model's generalization ability, particularly in crowded scenes, it aligns with the primary objective of this project: generating CAMs that clearly highlight individual objects. Training on less cluttered images may help the model learn better spatial representations and produce more interpretable CAMs. Figure 4 presents the class-by-class cumulative count of images as the number of annotations increases.

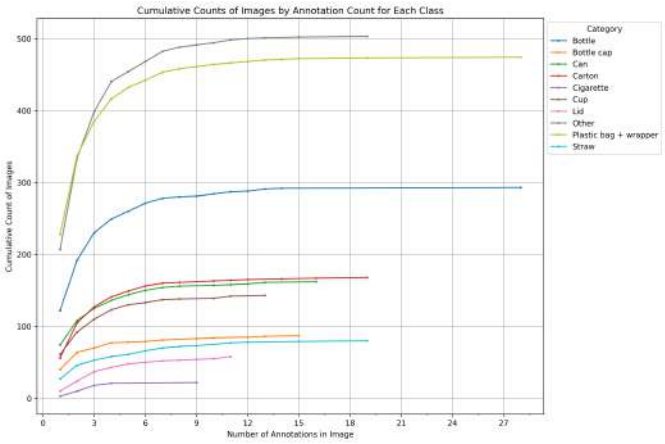


Figure 4: Per class cumulative count of images by annotation count.

The curves indicate that, for most classes, relatively few images contain more than 9–10 objects. Therefore, images with more than ten instances, such as the example shown in Figure 5, are removed.

Figure 5: Sample image with 28 annotations.

The remaining part of the dataset contains 1,375 unique images with a total of 2,516 annotations. The class distribution across images is illustrated in Figure 6, which shows a significant class imbalance. The majority of images contain instances of *Other* and *Plastic bag + wrapper*, while far fewer images include *Cigarette*, *Lid*, or *Straw*. A similar trend is observed in the distribution of annotations.

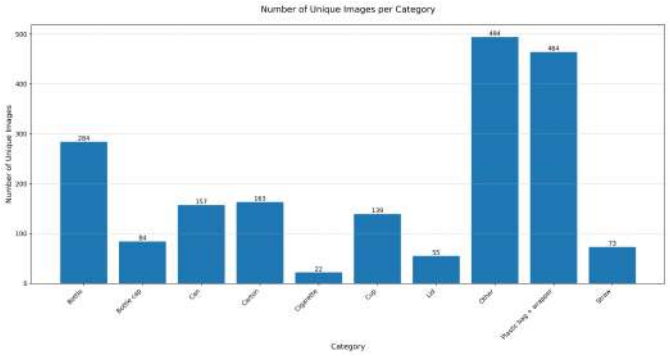


Figure 6: Category distribution across images.

In the next section, I describe an approach to mitigate this imbalance by generating new samples for the most underrepresented classes.

III. Synthetic Data Generation

The problem of balancing the dataset is particularly challenging for two key reasons. First, there is a substantial disparity between the most represented class (*Other*, with 494 images) and the least represented one (*Cigarette*, with only 22 images), making it impractical to upsample the latter while maintaining sufficient variability. Second, since the classification task is multi-label, undersampling must be performed carefully to avoid inadvertently removing samples of underrepresented classes when reducing instances of the more frequent ones.

It is important to note that the focus is on balancing class presence across images (as shown in Figure 6), rather than the number of annotations per class, as the latter is not relevant to the model's training objective.

The proposed approach, inspired in part by [4], involves generating synthetic samples for the underrepresented classes by extracting objects from existing images and pasting them onto randomly selected background images. These backgrounds are sourced from the original TACO repository and consist of various types of floors with similar depth. This strategy aims to create realistic samples by ensuring objects are placed in plausible locations, avoiding unnatural placements such as in the sky or on a person.

The four most underrepresented classes are selected for augmentation: *Cigarette*, *Lid*, *Straw*, and *Bottle cap*. For each of these classes, objects are extracted from images that contain only instances of the respective class. This ensures that the extracted objects are not overlapped or partially occluded by objects from other classes.

Once the pool of extracted objects is obtained, new images are generated by randomly selecting a background image and up to three objects, which may belong to different

underrepresented classes. Each selected object is randomly rotated and scaled, ensuring that its relative bounding box area (with respect to the background image) falls within the 1st and 3rd quartile of that class's relative bounding box area distribution. This prevents unrealistic object sizes in the synthetic samples. The objects are then randomly placed onto the background while minimizing excessive overlap.

The target number of total samples for each class is set to 160, aligning with the sample counts of other classes such as *Can*, *Carton*, and *Cup*. Given the limited number of background images (47) and extracted objects, generating a significantly larger dataset could result in overly similar samples, increasing the risk of overfitting.

Finally, before saving, random brightness, contrast, and saturation adjustments are applied to each new image to introduce additional variance and improve generalization.

Figure 7 presents an example of a synthetically generated image for each of the four augmented classes.

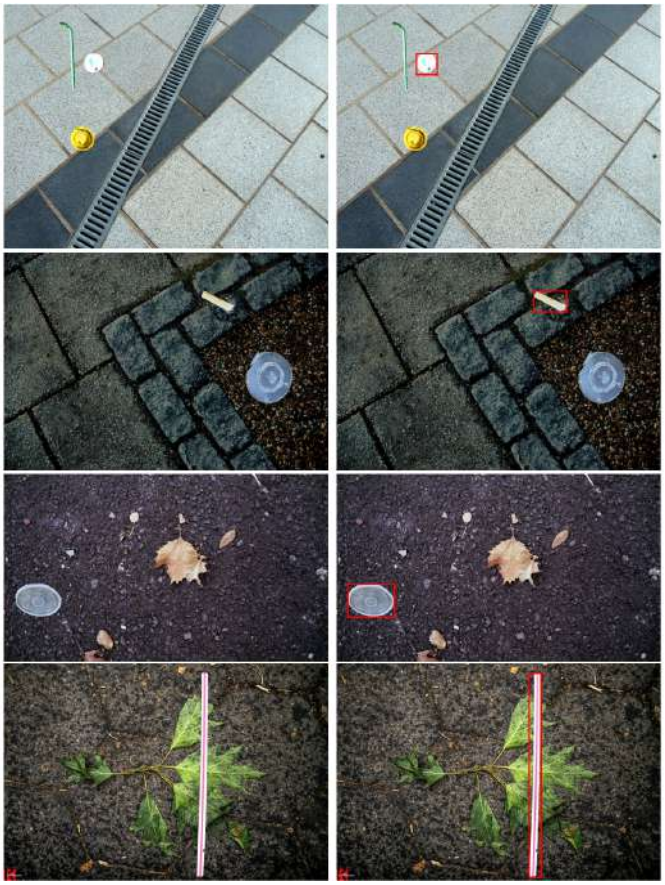


Figure 7: An example of synthetic image for each augmented class.

It can be noted as some objects naturally blend into the background and appear quite realistic, while others seem more "out of place," primarily due to differences in lighting conditions or perspective inconsistencies.

In a standard supervised pipeline, a portion of the original dataset should be reserved as a test set before applying any augmentation, simulating a real-world scenario where new data is received. However, in this project, synthetic images are generated using the entire dataset before splitting into training, validation, and test sets. This decision is driven by the scarcity of class-exclusive images, especially for the *Cigarette* class, where reserving a portion for testing could result in an insufficient number of samples for augmentation. To prevent biased results, such as the model encountering objects in the test set that it has already seen during training, the original images from which objects were extracted will be removed from the test and validation sets.

No undersampling is performed, as a substantial amount of data has already been discarded. While this operation could create a fully balanced dataset and potentially improve classification performance, it might also lead to data scarcity, preventing the network from learning meaningful representations necessary for accurate object localization.

The final class distribution across images is illustrated in Figure 8.

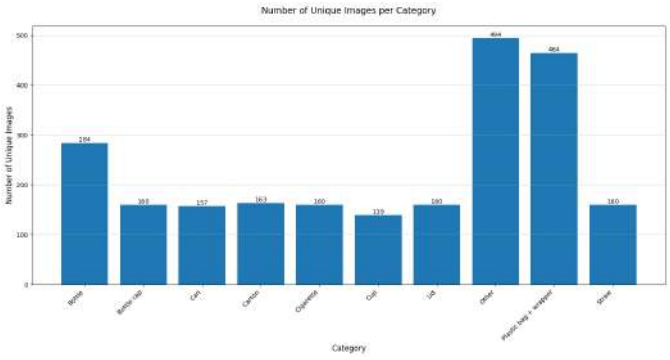


Figure 8: Class distribution across images after synthetic data generation.

IV. Data Split and Preparation

The dataset is divided into training (70%), validation (15%), and test (15%) sets using a stratified split, ensuring that each class is proportionally represented across all three subsets. This approach allows for a more reliable evaluation of the model's performance on each waste category.

As previously mentioned, images used to extract objects for synthetic data augmentation are removed from the validation and test sets to ensure fair model evaluation.

All images are resized to a standard resolution of 256×256, with bounding boxes for test images adjusted accordingly. To improve generalization, on-the-fly data augmentations such as random horizontal and vertical flips, color jittering, and random rotations are applied to training images. Addi-

tionally, all images are normalized using ImageNet standard values.

For the multi-label classification task, labels are encoded as binary 0-1 vectors of length equal to the number of classes, where 1 indicates the presence of a class in an image and 0 indicates its absence.

Finally, samples within each set are grouped into mini-batches, with different batch sizes tested during experimentation to balance memory constraints and generalization ability. The optimal batch sizes are found to be 16 or 32.

V. Methodologies

This section presents the two methods used to generate Class Activation Maps at test time. Class Activation Maps (CAMs) are a visualization technique used to highlight the regions of an image that contribute the most to a model's prediction. Because of their ability to provide insights into what the model focuses on when making predictions, they are considered Explainable AI (XAI) techniques.

In the context of Weakly Supervised Object Localization, CAMs play a crucial role in localizing objects without requiring precise bounding box annotations. Instead, they leverage the model's learned feature representations to infer the approximate locations of objects based only on image-level labels. In fact, by relying on CAMs, we can generate heatmaps that indicate the most relevant image regions for a given class prediction, helping to localize objects in a weakly supervised manner.

The original CAM method [5] requires modifications to the network architecture, such as replacing fully-connected layers with global average pooling (GAP). However, alternative approaches like Grad-CAM and LayerCAM have been developed to overcome these limitations. The following sections describe these two methods in detail and explain how they are applied in this study.

A. GradCAM

Gradient-weighted Class Activation Mapping (Grad-CAM) [6] is a widely used method for generating Class Activation Maps without requiring architectural modifications to the network or re-training. It extends the concept of CAMs to any CNN-based network by using the gradients of a target class with respect to the feature maps of a convolutional layer. This allows Grad-CAM to highlight image regions that contribute the most to the model's prediction. It has been proven in [6] that Grad-CAM is a strict generalization of the CAM method.

Let A^k represent the activation map of the k-th filter in a selected convolutional layer. The goal of Grad-CAM is to compute a heatmap $L^c_{Grad-CAM}$ that highlights important regions for a given class c. This is achieved through the following steps.

First, we compute the gradients of the predicted score y^c (before the sigmoid) with respect to the feature map activations, i.e. $\frac{\partial y^c}{\partial A^k}$. These gradients indicate how much each activation map contributes to the class score.

Next, the gradients are averaged spatially (GAP over the width and height dimensions) to obtain importance weights for each feature map for class c:

$$\alpha^c_k = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A^k_{ij}}.$$

where Z is the number of spatial locations in the activation map.

The final activation map is obtained by computing a weighted sum of the forward activation maps followed by a ReLU operation:

$$L^c_{Grad-CAM} = ReLU\Big(\sum_k \alpha^c_k A^k\Big).$$

The ReLU function ensures that only positive contributions are considered, as negative activations are likely to correspond to irrelevant or background regions.

The resulting heatmap is upsampled to the original image size and then normalized between 0 and 1 to produce the final visualization.

Grad-CAM effectively highlights discriminative regions of an image, making it useful for object localization in WSOL. However, since it relies on the output of the final convolutional layer, which typically has low spatial resolution, it often produces coarse heatmaps, which may fail to capture fine details of objects, particularly when they are small or scattered. To address this limitation, LayerCAM introduces a more refined approach, as discussed next.

B. LayerCAM

To overcome the limitations of Grad-CAM, LayerCAM was introduced in [7]. This method exploits class activation maps from shallow convolutional layers, which have higher spatial resolution, to achieve more accurate and fine-grained object localization. Unlike Grad-CAM, which assigns a global importance weight to each feature map, LayerCAM computes location-specific importance weights based on the gradients, allowing it to better distinguish objects from the background.

In fact, a key issue when applying Grad-CAM to shallow layers is that a single global weight per feature map fails to eliminate the fine-grained noisy details captured in the background, leading to inaccurate localization.

LayerCAM addresses this by computing a unique weight for each spatial location (i, j) in the k-th feature map:

$$w_{ij}^{kc} = ReLU\big(g_{ij}^{kc}\big).$$

where $g_{ij}^{kc} = \frac{\partial y^c}{\partial A_{ij}^k}$ represents the gradient of the class score y^c with respect to the feature activation at location (i, j).

Next, each activation value in the k-th feature map is multiplied by its corresponding weight, obtaining $\hat{A}_{ij}^k$:

$$\hat{A}_{ij}^k = w_{ij}^{kc} \cdot A_{ij}^k.$$

The final class activation map is then computed by linearly combining the weighted activation maps along the channel dimension, followed by a ReLU operation:

$$L_{LayerCAM}^c = ReLU\big(\textstyle\sum_k \hat{A}^k\big).$$

This method retains more spatial information from the feature maps, making it particularly effective for detecting smaller objects, such as litter in the TACO dataset.

A key advantage of LayerCAM is that it allows the combination of CAMs from different layers to generate more precise and complete object localization. To do this, each individual class activation map is first scaled as described in [7], using the function:

$$\tanh\left(\frac{\gamma \cdot L_{LayerCAM}^c}{\max\big(L_{LayerCAM}^c\big)}\right).$$

where γ is a scaling factor.

Finally, the min-max normalization is applied to each map, and the activation maps from different layers are combined using an element-wise maximum operation, ensuring that the final CAM retains the most salient features across multiple layers.

The discussed methodologies are applied at test time to generate CAMs for the trained model, allowing for qualitative and quantitative evaluation of its localization performance. The next section discusses the training and validation processes used to prepare the model for this evaluation.

VI. Training and Validation

To perform the multi-label classification task, a VGG-16 network pretrained on ImageNet is fine-tuned [8]. The last fully-connected layer is modified to have 10 output neurons (equal to the number of target classes) instead of 1000, as in the original ImageNet model.

In addition to the standard VGG-16 architecture, which has approximately 134M parameters, a simplified version is introduced by reducing the number of neurons in the fully-connected layers. This smaller model contains around 68M parameters, helping to mitigate overfitting, which arises due to the complexity of the network compared to the relatively small size of the dataset. To further reduce overfitting, batch normalization layers are added to the classifier part of both networks.

Despite VGG-16 being overly complex for the TACO dataset, it was chosen because of its strong feature extraction capabilities, which are particularly useful for the WSOL task.

Different fine-tuning strategies are explored during experimentation. These include training only the classifier while keeping all convolutional layers frozen, initializing training with a frozen convolutional base and progressively unfreezing the last convolutional blocks after a few epochs, and training some convolutional blocks from the beginning.

The loss function used for training is Binary Cross-Entropy (BCE) with Logits, which combines the sigmoid activation function and the binary cross-entropy loss into a single computation.

For a given prediction $\hat{\mathbf{y}} \in \mathbb{R}^C$ and the corresponding target $\mathbf{y} \in \mathbb{R}^C$, the loss $L(\mathbf{y}, \hat{\mathbf{y}})$ is defined as

$$\tfrac{1}{C} \textstyle\sum_c -w_c[y_c \log \sigma(\hat{y}_c) + (1 - y_c) \log(1 - \sigma(\hat{y}_c))],$$

where σ denotes the sigmoid function and w_c represents an optional class weight.

Both weighted and non-weighted loss functions are tested. When using weighted loss, class weights are computed as the inverse of class occurrence across training images, normalized so that their sum equals the number of categories. This weighting strategy aims to compensate for class imbalances by assigning higher importance to underrepresented classes.

The maximum number of training epochs is set to 30, though in most cases, the model exhibits signs of overfitting before reaching this limit. An early stopping criterion is applied, where training is interrupted if the validation loss does not improve beyond a certain threshold for five consecutive epochs.

Training is conducted with the Adam optimizer and a learning rate scheduler that monitors the validation loss and reduces the learning rate by a factor of 0.5 after three epochs of non-improvement, allowing the model to adjust its learning parameters before training is stopped.

An initial learning rate of $1 \cdot 10^{-4}$ is assigned to the classification head, while a lower learning rate is used for the convolutional layers to prevent catastrophic forgetting of

pretrained features. Weight decay is implemented as a regularization technique to prevent overfitting, acting as an L_2 penalty on the model's parameters, discouraging excessively large weights. This is particularly beneficial given the small dataset size, which could otherwise cause the network to memorize the training examples rather than learning generalized patterns.

Validation is performed at the end of each epoch. During this phase, the training and validation losses are recorded, alongside the Mean Average Precision (mAP). The mAP is a commonly used metric for evaluating multi-label classification performance. It measures how well the model ranks correct classes higher than incorrect ones across multiple images. Formally, for each class c, the Average Precision (AP) is computed as

$$AP_c = \sum_n (R_n - R_{n-1}) P_n$$

where P_n and R_n are the precision and recall at the n-th threshold, respectively. mAP is simply obtained by averaging the AP scores over all classes.

VII. Results

This section presents the evaluation of the trained models on both waste detection and waste localization tasks. The effectiveness of different training strategies is analyzed, including the impact of incorporating synthetic data, varying fine-tuning approaches, and using different loss weighting schemes.

A. Waste Detection

To assess the model's ability to correctly classify multiple waste categories in an image, the mean F1 Score (mF1) and mAP are used as metrics. While mAP measures the ranking quality of the predicted class scores, the F1 Score provides a balance between precision and recall. The F1 Score for a single class is defined as:

$$F1 = \frac{2\times \text{TP}}{2\times \text{TP} + \text{FP} + \text{FN}}$$

where TP, FP, and FN denote true positive, false positive, and false negative predictions, respectively.

The mean F1 Score is obtained by averaging the scores across all classes. The class-by-class metrics are also saved and analyzed.

The baseline model, consisting of the original VGG-16 trained by fine-tuning only the classification head without the addition of synthetic data, achieves a mF1 of 20.55% and a mAP of 25.12% on the test set. However, its performance on underrepresented classes remains notably poor, with mAP values below 5% for *Bottle cap*, *Cigarette*, *Lid*, and *Straw*.

By incorporating synthetic data into training, the model exhibits a significant improvement, reaching a mF1 of 24.69% and a mAP of 28.48%. This enhancement is particularly noticeable for the *Bottle cap* and *Straw* classes, whose mAP values double compared to the baseline.

The highest detection performance is obtained when using the simplified version of VGG-16, with the last two convolutional blocks unfrozen after five epochs. This model achieves a mF1 of 31.04% and a mAP of 32.92%, with a substantial improvement in detecting underrepresented categories. Particular improvement is seen for the *Bottle cap* class, which reaches an AP over 36%, while the other minority classes only show slight gains, as shown in Figure 9.

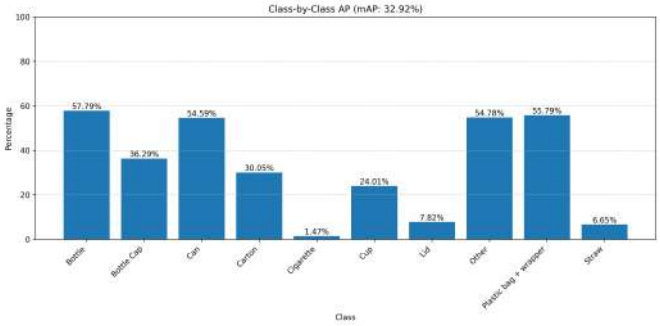


Figure 9: Class-by-class AP for simplified VGG-16.

Despite these improvements, the *Cigarette* class consistently records the lowest performance, with an AP never exceeding 3%. This is partially due to the fact that the test set contains only two images featuring cigarettes, meaning that a single misclassification has a disproportionately large effect on the final score. Finally, training with a weighted loss does not lead to meaningful improvements for underrepresented classes. Instead, it negatively impacts the detection of more frequent categories, making the trade-off unfavorable in this context.

Table 1 summarizes the key results obtained for waste detection across different models.

Model	**Synthetic Data**	**Fine-Tuning**	**mF1**	**mAP**
Original VGG-16	No	Classifier only	20.55%	25.12%
Original VGG-16	Yes	Classifier only	24.69%	28.48%
Original VGG-16	Yes	Last Conv Blocks	23.42%	29.96%
Simplified VGG-16	Yes	Last Conv Blocks	31.04%	32.92%

Table 1: Main waste detection results.

B. Waste Localization

To evaluate object localization, the mean Intersection over Union (mIoU) is used. This metric quantifies the overlap between the predicted and ground truth bounding boxes, providing insight into the model's ability to precisely localize objects.

Bounding boxes are extracted from the generated CAMs using two different strategies. The first method applies a fixed threshold to binarize the CAM, followed by contour extraction and filtering of small regions below a fixed area threshold to reduce noise. The second method refines this approach by first selecting only the most confident regions (threshold = 0.9) and then expanding the bounding boxes by incorporating additional regions from a lower-thresholded map. This second strategy helps define more accurate bounding boxes by focusing on high-confidence regions first and then expanding them carefully.

Various area thresholds are explored to filter out small, irrelevant detections. A fixed area threshold of 1000 pixels is found to be the most effective. While adjusting the threshold for each class based on the relative size distribution of its bounding boxes is also tested, this approach does not yield meaningful improvements, likely due to an increased retention of incorrect detections.

Different combinations of CAMs from various layers are tested for LayerCAM. Both quantitative and visual results indicate that the best performance is achieved when excluding the class activation map from the last convolutional block (i.e., from layer 'features.30') and instead using the ones from the two preceding blocks (i.e., from layers 'features.23' and 'features.16'). This is because, as mentioned before, earlier layers capture finer details, while deeper layers tend to focus on more abstract and larger-scale features.

In particular, the CAM from the last convolutional layer typically highlights broader regions, as it captures high-level semantic information rather than precise object boundaries. In contrast, CAMs from earlier layers tend to emphasize smaller, more localized regions, as shown in Figure 10, making them more suitable for detecting fine-grained details. Given that most objects in the dataset are relatively small, incorporating the last-layer CAM results in excessive activation over large areas, reducing localization precision. Therefore, relying on the activation maps from intermediate layers leads to better object delineation and improved localization accuracy.

The scaling factor γ is set to 2, as suggested in [7]. The best threshold to extract bounding boxes from LayerCAM is found to be 0.8, while for Grad-CAM, the optimal threshold typically lies between 0.5 and 0.6.

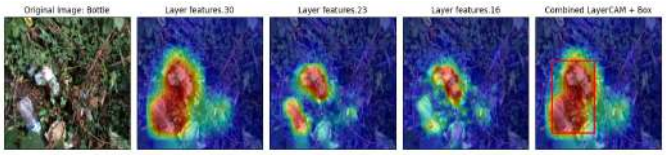


Figure 10: Class activation maps from different layers of VGG-16.

The impact of different fine-tuning strategies is also evaluated. The baseline model, trained only on the classifier with frozen convolutional layers, achieves a mIoU of 16.29% with Grad-CAM and 23.98% with LayerCAM when trained without synthetic data. When synthetic data is included in training, these values increase to 18.31% with Grad-CAM and 25.62% with LayerCAM, confirming the beneficial role of data augmentation.

Further performance gains are observed when unfreezing convolutional layers. The best-performing model, where the last two convolutional blocks are unfrozen after three epochs, achieves a mIoU of 19.27% with Grad-CAM and 28.52% with LayerCAM, highlighting the advantage of allowing feature representations to be refined. Figure 11 illustrates the class-by-class IoU for this last case.

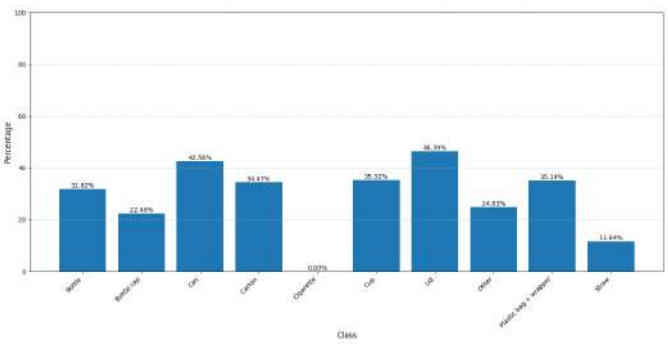


Figure 11: Class-by-class IoU of best model (using LayerCAM).

It can be observed that the localization performance for the *Cigarette* class is virtually nonexistent, while other underrepresented classes, such as *Lid*, achieve significantly better results. This is likely due to the fact that *Lid* instances are generally larger than *Cigarette* objects, making them easier to detect. Moreover, small cigarette objects tend to produce weaker activations in the CAMs and may be overshadowed by background noise or merged with surrounding regions. Additionally, their bounding boxes might be discarded due to the minimum area requirement.

While the second bounding box extraction strategy (high-confidence filtering before expansion) usually yields slight improvements, its impact on localization accuracy remains limited. This suggests that most performance gains result

from improving the CAM quality rather than the specific bounding box extraction method.

Table 2 presents the localization results for different models and CAM extraction strategies.

Model	Synthetic Data	Fine-Tuning	Grad-CAM mIoU	Layer-CAM mIoU
Original VGG-16	No	Classifier only	16.29%	23.98%
Original VGG-16	Yes	Classifier only	18.31%	25.62%
Original VGG-16	Yes	Last Conv Blocks	19.27%	28.52%
Simplified VGG-16	Yes	Last Conv Blocks	16.27%	25.97%

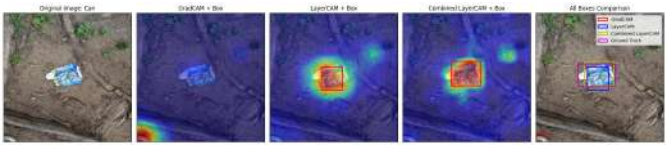
Table 2: Main waste localization results.

As expected, LayerCAM outperforms Grad-CAM across all experiments.

Figure 13 displays examples of class activation maps obtained using Grad-CAM, LayerCAM from the last convolutional layer, and the combined LayerCAM from intermediate layers. The last column presents the predicted bounding boxes alongside the ground truth annotations for the class of interest.

It can be observed that the bounding boxes extracted from the combined LayerCAM are generally more accurate, which aligns with the numerical results. The model appears to struggle particularly with the *Other* class, likely because this category includes a diverse range of objects with highly varying features, making it difficult for the network to learn distinctive representations.

Another important observation is that, while the extracted bounding boxes are often imprecise, highlighting the challenge of designing an optimal extraction strategy, the class activation maps indicate that the network frequently attends to the correct regions. For instance, when identifying a bottle, the network often focuses on relevant elements such as the bottle neck and the bottle cap, even if the final bounding box does not perfectly enclose the object.



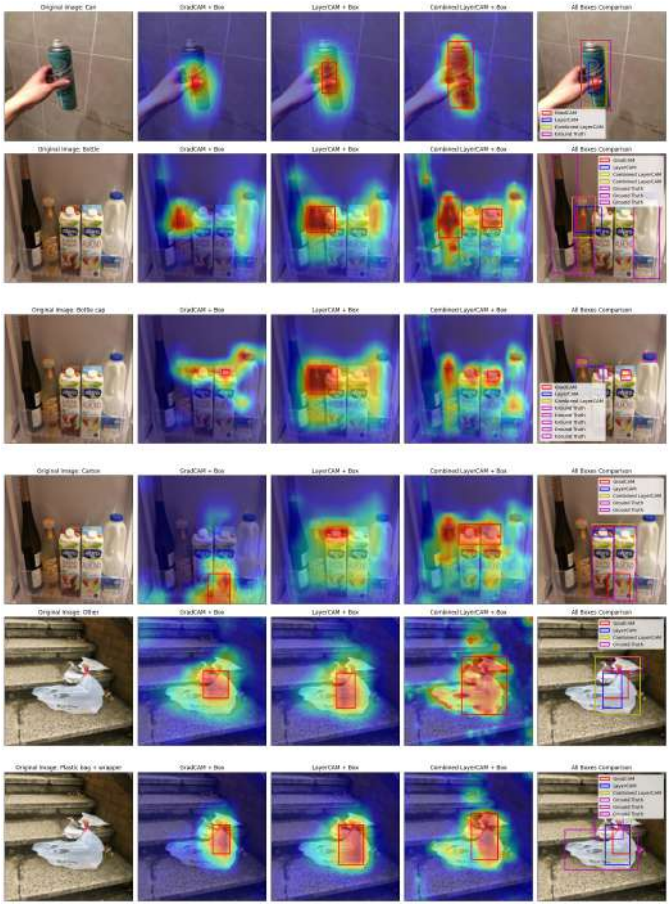


Figure 13: Examples of CAMs and extracted bounding boxes.

VIII. Conclusions

In this work, a weakly supervised object localization (WSOL) approach was applied to the task of multi-label waste detection using a VGG-16-based model fine-tuned on the TACO dataset. The primary objective was to evaluate the model's ability to detect and localize different types of litter using class activation maps. Several fine-tuning strategies were explored, and the impact of synthetic data augmentation was analyzed. The best classification performance was achieved using a simplified version of VGG-16 with the last two convolutional blocks unfrozen after five epochs, yielding a mean F1 Score of 31.04% and a mean Average Precision (mAP) of 32.92%. For localization, the highest accuracy was obtained using LayerCAM with activation maps from intermediate layers, reaching a mean Intersection over Union (mIoU) of 28.52%.

The results highlight the significant challenges posed by this task. The variety of objects, backgrounds, and lighting conditions contributes to the complexity, but the primary difficulty arises from the nature of the dataset. The vast majority of waste instances are very small compared to the

overall image size, making their detection and localization particularly challenging. Furthermore, the dataset is highly unbalanced, with certain categories being significantly underrepresented, which negatively impacts the model's ability to learn robust features for all classes. This is particularly evident for the *Cigarette* category, which consistently achieves the lowest performance across all experiments.

Synthetic data generation proved to be an effective strategy, leading to notable improvements in both classification and localization performance. The use of synthetic images increased the mAP from 25.12% to 28.48% when training only the classifier and improved localization performance from a mIoU of 23.98% to 25.62% when using LayerCAM. However, further refinements could improve its effectiveness. In particular, incorporating more realistic variations in illumination, object scale, and background selection could improve generalization to real-world scenarios.

Another potential avenue for improvement is related to image resolution. Due to computational constraints, the images were significantly resized, which likely resulted in a loss of important fine-grained details, especially for small objects. If sufficient memory and processing power were available, training with higher-resolution images could lead to better performance. Alternatively, a tiling strategy could be explored, where images are divided into smaller patches for training and inference, ensuring that the label information is appropriately maintained.

Finally, dataset balancing remains an open issue. While weighting the loss function did not yield improvements, undersampling the most frequent categories could be explored as a potential way to enhance classification performance for underrepresented classes. Future work should focus on these aspects to further refine the model and improve both litter detection and localization accuracy.

References

[1] "Litter Study," [Online]. Available: https://kab.org/litter/litter-study/

[2] "Data Collection on Costs of Littering and Anti-littering Strategies as a Part of EPR Schemes," [Online]. Available: https://www.eprclub.eu/events/data-collection-on-costs-of-littering-and-antilittering-strategies-as-a-part-of-epr-schemes/

[3] "Environment: Over 80% of Litter on Italian Beaches is Plastic," [Online]. Available: https://www2.enea.it/en/news-enea/news/environment-over-80-of-litter-on-italian-beaches-is-plastic

[4] P. F. Proença and P. Simões, "TACO: Trash Annotations in Context for Litter Detection." [Online]. Available: https://arxiv.org/abs/2003.06975

[5] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning Deep Features for Discriminative Localization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016.

[6] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations From Deep Networks via Gradient-Based Localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct. 2017.

[7] P.-T. Jiang, C.-B. Zhang, Q. Hou, M.-M. Cheng, and Y. Wei, "LayerCAM: Exploring Hierarchical Class Activation Maps for Localization," *IEEE Transactions on Image Processing*, vol. 30, no. , pp. 5875–5888, 2021, doi: 10.1109/TIP.2021.3089943.

[8] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition." [Online]. Available: https://arxiv.org/abs/1409.1556